

08 — Infrastructure & Operations

Objetivo: documentar la plataforma sobre la que corre Puglit — el hardware, la GPU, la topología de contenedores, los gateways de soporte, el storage, la red, el monitoreo, el logging, y los procedimientos operativos de backup, disaster recovery y upgrade. Todo verificado contra el código real en `/Users/alvaroz/projects/2026/puglit` (carpeta `infra/`, `web/scripts/build-local.mjs`, `web/lib/brain-sync.ts`, `web/lib/db.ts`).

1. Infrastructure Overview

Puglit es **self-hosted y agent-first**: el sistema genético multi-agente que genera apps Next.js 16 + Postgres no depende de ninguna API paga. Corre **100% local sobre Ollama** (sin API key, sin costo), con la opción de traer (BYO) cualquier proveedor OpenAI-compatible o Anthropic. Toda la inteligencia — los 75 agentes, el torneo genético, el cerebro evolutivo — vive en una sola caja con GPU.

La forma canónica de la plataforma es una **GPU box única** (un RunPod Pod o una VM con GPU) que concentra cuatro cosas:

1. **Ollama** sirviendo los modelos de código y auxiliares (qwen2.5-coder:32B + gemma2:9B + embeddings).
2. **Postgres 14 + pgvector** como almacén autoritativo: jobs, proyectos, y el "cerebro" (diary, métricas, exemplars, skills, módulos).
3. La **web app Next.js** (`web/`) que orquesta el swarm — interview → spec → blueprint → routes/pages → supervisiones, más el torneo genético y el evolution engine.
4. Un conjunto de **sidecars / gateways** opcionales (scraping, OCR, PDF, search, notifications, OAuth, workflows) que dan capacidades a los módulos inyectados en las apps generadas.

El diseño nace de una restricción concreta y honesta, documentada en `infra/setup-gpu-box.sh`: **una laptop de 8GB no aguanta el stack**. Un modelo de 7B "oscila" (entra en loops, no converge en código). Por eso todo el stack se mueve a una caja con GPU real para correr un modelo de código capaz (qwen2.5-coder:32B) en lugar del 7B.

```
> Moves the WHOLE local stack off an 8GB laptop onto a real GPU so the agents
> run a capable code model (qwen2.5-coder:32B) instead of the 7B that oscillates.
```

El setup es **one-shot e idempotente**: `bash infra/setup-gpu-box.sh` instala Ollama + modelos, Postgres, Node 20, las deps del repo, el schema (`web/sql/puglit.sql` + `web/sql/genetic.sql`), siembra el roster de 75 agentes (`POST /api/genetic/seed`), y deja Ollama + el dev server corriendo. Es seguro re-correrlo.

Two deployment shapes

Shape	Dónde corre	systemd	Notas
RunPod Pod (recomendado, más barato)	Contenedor Ubuntu+CUDA, normalmente root	No	El setup arranca Ollama vía <code>nohup ollama serve</code> y Postgres vía <code>pg_ctlcluster</code> ; expone puertos por el RunPod proxy
Paperspace / DigitalOcean Machine	VM Ubuntu completa	Sí	El setup usa <code>systemctl</code> ; se accede por túnel SSH

El script de setup detecta el caso automáticamente (`SUDO=""` cuando es root, helpers `start_pg()` que prueban `systemctl` → `service` → `pg_ctlcluster` en cascada).

Existe además un **objetivo de deploy diferente** que NO debe confundirse con la GPU box: las **apps generadas** se entregan al usuario en SU propia cuenta (BYO deploy a GitHub + Vercel opcional, `infra/deploy.sh`) y el spine trae un `infra/fly.toml.template` + `infra/Dockerfile` para empaquetar cada app generada como imagen standalone Next.js. Eso es infraestructura *del producto generado*, no de la plataforma Puglit. Esta sección documenta la plataforma; la sección de deploy de apps se cubre aparte.

2. Hardware Requirements

La caja de referencia, documentada en la memoria de infra y en los headers de los scripts, es un **RunPod A40 de 48GB de VRAM**. El factor limitante real es la **VRAM** (para cargar el modelo de 32B sin caer a CPU) y el **disco persistente** (para no re-descargar ~77GB de modelos en cada arranque).

Plataforma (GPU box)

Recurso	Requisito	Por qué
GPU VRAM	≥ 48GB (A40 de referencia)	qwen2.5-coder:32B @ Q4 ocupa ~20GB; el resto deja headroom para gemma2:9B + embeddings + KV cache
Modelos en disco (Ollama)	~77GB	qwen2.5-coder:32B (~20GB) + gemma2:9B + nomic-embed-text + opcionalmente un modelo de visión
Volume / disco persistente	100GB	Modelos (~77GB) + node_modules del spine + builds (<code>.builds/</code>) + datos de Postgres
RAM (host)	Suficiente para build + un modelo	El build de Next corre antes de cualquier inferencia para que el 32B no esté en RAM y no haya OOM (ver nota en <code>setup-gpu-box.sh</code> paso 7)
CPU	Multi-core estándar	tsc, Postgres, Next dev/start, los sidecars Python
Node	≥ 20	El setup instala Node 20 vía NodeSource si falta
Postgres	14 + pgvector	Almacén autoritativo de jobs/proyectos/cerebro; pgvector para el módulo <code>rag</code> y los embeddings del diary

Nota crítica de hardware (literal en `setup-gpu-box.sh`): `pciutils` (`lspci`) + `lshw` **deben** instalarse ANTES de Ollama, o Ollama no detecta la GPU y silenciosamente instala la versión CPU-

only — y un 32B en CPU "crawls". El setup los instala primero por esa razón.

Tunables del setup (env overrides)

```
MODEL_CODE=qwen2.5-coder:32b    # lane code + premium + judge (~20GB VRAM @ Q4)
MODEL_AUX=gemma2:9b             # lane balanced + cheap (extracción, diversidad MoA)
MODEL_VISION=                   # opcional, e.g. qwen2.5vl:7b → habilita referencias por imagen
PG_DB=puglit PG_USER=postgres PG_PORT=5432 DEV_PORT=3000
```

Económica operativa

El modelo de costos es explícito en el cierre del setup: **se paga compute solo mientras el pod está encendido**; el volume disk persiste los modelos para el próximo arranque. Por eso el patrón operativo recomendado es: encender → trabajar → **STOP** el pod. Snapshots del cerebro (sección 10) garantizan que apagar no pierda aprendizaje.

3. GPU Requirements

La GPU es el corazón de la plataforma. Tres reglas operativas, todas presentes en el código:

1. **Detección de GPU antes de instalar Ollama.** Sin `pciutils / lshw`, Ollama se instala CPU-only. El setup garantiza el orden.
2. **Flash Attention activado.** Ollama se arranca con `OLLAMA_FLASH_ATTENTION=1` (en `setup-gpu-box.sh` y `rebuild.sh`) para acelerar la atención y reducir el footprint del KV cache.
3. **Un modelo cargado a la vez para evitar OOM en swaps.** En `rebuild.sh` el council de Ollama corre con `OLLAMA_MAX_LOADED_MODELS=1`: cuando los jueces / agentes alternan entre el modelo de código y el auxiliar, no se intenta tener los dos residentes a la vez. El build de la web corre con Ollama abajo (modelo fuera de RAM) → sin OOM durante el build.

Aceleración opcional (documentada, no obligatoria)

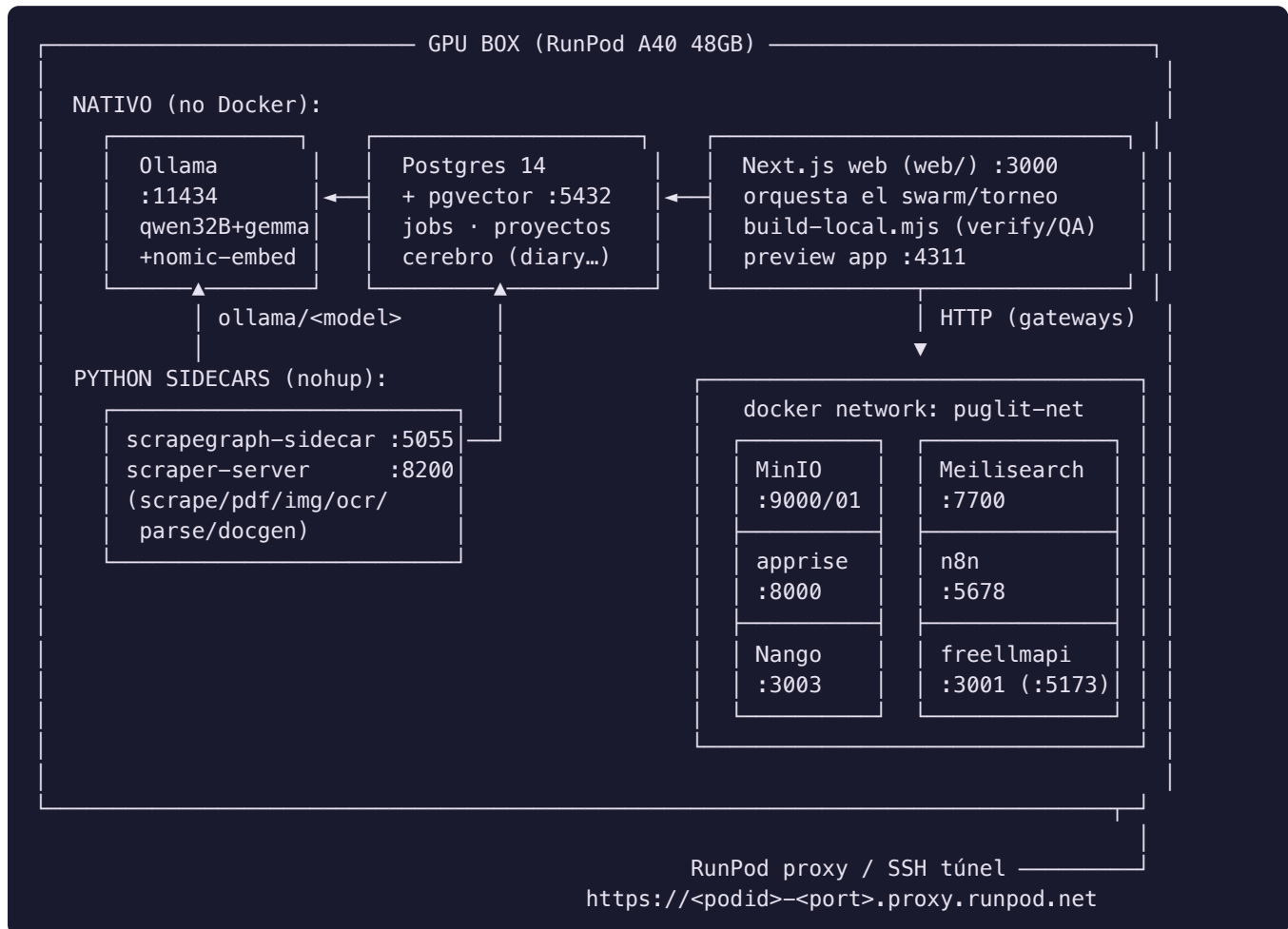
`infra/llm-acceleration.md` describe dos optimizaciones de **infra pura** (cero cambios al código de generación), porque el swarm reusa los MISMOS system prompts (architect/dev/critic) en cada agente → hit-rate de cache altísimo, especialmente en el panel de jueces (3 prompts con prefijo común en paralelo):

- **LMCache + vLLM** — KV-cache de prompts repetidos. Se apunta `PUGLIT_CODE_BASE_URL` al puerto de vLLM (mismo OpenAI-compatible, sin cambios de código).
- **freellmapi** — proxy OpenAI-compatible que agrega 16+ providers de tier gratis con failover, dando acceso a modelos MÁS grandes que los que entran en el A40. Con el caveat honesto de que **no hay frontier ni SLA**: se usa para volumen, con el **A40 local como fallback confiable**, y el judge/critic se mantiene en el modelo local consistente (no se lo deja rotar mid-scoring).

Ambas se cablean por env por tier (`PUGLIT_MODEL_*`, `PUGLIT_*_BASE_URL`, `PUGLIT_*_API_KEY`) — la abstracción de proveedor ya las lee.

4. Docker Topology

La plataforma Puglit core (Ollama + Postgres + web) corre **nativa en la caja** (no containerizada): es lo que instala `setup-gpu-box.sh`, para minimizar overhead y dejar la GPU al modelo. Lo que **sí** corre en Docker son los **gateways de soporte**, levantados de una por `infra/setup-gateways.sh` sobre una red bridge dedicada `puglit-net`.



`infra/setup-gateways.sh` crea la red (`docker network create puglit-net`) y levanta cada contenedor con `--network puglit-net`, mapeo de puertos al host, y un volumen nombrado para la persistencia. Cada `docker run` hace `docker rm -f` previo del contenedor homónimo → **idempotente**.

Para la imagen de las **apps generadas** existe `infra/Dockerfile`: build multi-stage (deps → build → runtime) de Next.js standalone, `EXPOSE 3000`, `CMD ["node", "server.js"]`. Es la plantilla que empaqueta lo que el generador produce, no la plataforma.

5. Gateway Architecture

Los gateways son servicios separados para que las apps generadas queden **thin** (sin Chromium ni dependencias pesadas dentro de la app): el módulo correspondiente sólo hace una llamada HTTP. Todos

usan el **Ollama local** para la parte de LLM (free, sin key, headless). Estos son los que existen de verdad en el repo:

Gateway	Archivo / origen	Puerto	Qué hace
ScrapeGraphAI sidecar	infra/scrapegraph-sidecar.py	5055	Reference grounding para el agente <code>researcher / reference-studier</code> (<code>POST /extract</code> de una URL competidora → entidades reales para que el architect diseñe desde la realidad) + el módulo <code>scrape</code> de las apps. Flask, SmartScraperGraph sobre <code>ollama/<model></code>
scraper-server (gateway omnibus)	infra/scraper-server.py	8200	FastAPI con seis endpoints: <code>/fetch</code> (Scrapling stealth, bypassa Cloudflare/anti-bot, renderiza JS), <code>/extract</code> (ScrapeGraphAI), <code>/pdf</code> (HTML→PDF vía Playwright/Chromium), <code>/image</code> (resize/optimize, Pillow), <code>/ocr</code> (Tesseract/pytesseract, eng+spa), <code>/parse</code> (cualquier doc → Markdown, microsoft/markitdown), <code>/docgen</code> (genera docx/xlsx)
MinIO	setup-gateways.sh	9000 / 9001	Storage S3-compatible (módulo de media/uploads)
Meilisearch	setup-gateways.sh	7700	Search para apps con búsqueda full-text
apprise-api	setup-gateways.sh	8000	Fan-out de notificaciones (email/Telegram/Slack/Discord/...)
n8n	setup-gateways.sh	5678	Workflows para módulos que orquestan automatizaciones
Nango	setup-gateways.sh	3003	OAuth proxy para conectores de terceros (requiere su propio Postgres vía <code>NANGO_DB_URL</code>)
freellmapi	setup-gateways.sh (clona + <code>docker compose</code>)	3001 (dashboard 5173)	Proxy OpenAI-compat de 16+ providers de tier gratis; LLM boost/fallback del swarm
pgvector	setup-gateways.sh	(extensión)	<code>CREATE EXTENSION vector</code> en el Postgres existente, para el módulo <code>rag</code> y embeddings del diary

`OpenWA` (WhatsApp) y `flux-server` (image-gen, `infra/setup-flux.sh` / `flux-server.py`) figuran como **opcionales y pesados** y no se levantan por defecto.

El wiring se hace por env en `web/.env.local` (`setup-gateways.sh` imprime las URLs al final): `PUGLIT_SCAPEGRAPH_URL` , `SCRAPER_URL` , `S3_ENDPOINT` / `S3_BUCKET` / `S3_*_KEY` , `MEILI_URL` / `MEILI_KEY` , `APPRISE_URL` , `N8N_URL` , etc. Si una env no está, el módulo correspondiente simplemente no se ofrece — degradación elegante, sin crashes.

Postura de seguridad de los sidecars: nunca devuelven 5xx al swarm. ScrapeGraphAI captura toda excepción y devuelve `{error, result:null}` con HTTP 200, para que el caller pueda hacer no-op en vez de tumbar el build. El header de `scraper-server.py` documenta la zona gris honesta (cookies de sesión para targets autenticados, respeto de rate limits, rotación de proxies, riesgo de bans).

6. Storage

Hay tres clases de storage, con límites de durabilidad distintos:

6.1 Postgres data (autoritativo, persiste en el volume)

El **almacén autoritativo** es Postgres. `web/lib/db.ts` abre un `Pool` de `pg` parametrizado por env (`POSTGRES_HOST/PORT/DB/USER/PASSWORD`, `POSTGRES_POOL_MAX` default 5, `application_name="puglit-web"`). En producción el host es el **pooler de Supabase (Supavisor)** y TLS es obligatorio; en la GPU box local se usa Postgres nativo con `POSTGRES_SSL=disable`. El pool tolera la falta de config: `isConfigured()` deja que la UI lo diga honestamente en vez de fingir.

Dos schemas conviven:

- **App / plataforma** (`web/lib/db.ts` `SCHEMA_SQL` + `web/sql/puglit.sql`): `puglit_projects`, `puglit_waitlist`, `puglit_jobs` (con `status`, `artifacts` JSONB, `steps`, `lease_until` para el watchdog, `user_email` para scoping multi-usuario, índices por status/usuario).
- **Genetic / cerebro** (`web/sql/genetic.sql`): `puglit_teams`, `puglit_agents` (75 agentes con stats RPG, level, xp, wins, quality), `puglit_agent_diary`, `puglit_metrics`, `verified_exemplars`, `puglit_modules`, `puglit_skills`, `puglit_skill_rejects`.

Fuera de producción, `ensureSchema()` corre el `SCHEMA_SQL` como conveniencia de dev (en prod el rol de app no puede DDL, así que el SQL se corre a mano una vez).

6.2 Build artifacts (efímero, en el volume mientras dura)

`web/scripts/build-local.mjs` materializa cada build bajo `.builds/puglit-<slug>/` **dentro del repo** (no en `/tmp`) por una razón concreta de Turbopack: un symlink de `node_modules` que cruza filesystems (`/tmp` → `/workspace`) hace panic. Por eso el build vive en el mismo filesystem que el spine y `node_modules` se materializa con `cp -al` (hard-links: instantáneo, sin disco extra, inodes compartidos), con fallback a `clonefile` / copia real.

Cada build usa además **su propia DB efímera** (`puglit_app-<slug>` en `PG_PORT` 5433 por defecto), que se hace `dropdb` + `createdb` en cada intento de carga de SQL (`resetDb()`), aislando el schema de la app generada del Postgres de la plataforma.

6.3 Modelos de Ollama (semi-persistente, en el volume)

Los ~77GB de modelos viven en el volume del pod (`OLLAMA_MODELS=/workspace/.ollama` en `rebuild.sh`). Es lo que justifica los 100GB de volume y el patrón "apagá el pod, el disco persiste los modelos".

6.4 Gateway data (Docker volumes nombrados)

MinIO (`puglit-minio:/data`), Meilisearch (`puglit-meili:/meili_data`) y n8n (`puglit-n8n:/home/node/.n8n`) persisten en volúmenes Docker nombrados sobre `puglit-net`.

7. Networking

Servicio	Puerto	Binding	Acceso
Ollama	11434	0.0.0.0 (OLLAMA_HOST=0.0.0.0)	Interno (web + sidecars)
Postgres (plataforma)	5432	localhost	Interno
Postgres (builds efímeros)	5433	localhost	Interno (build-local)
Next.js web	3000 (DEV_PORT)	0.0.0.0	RunPod proxy / túnel SSH
Preview de la app generada	4311 (PORT)	0.0.0.0	RunPod proxy (slot reusable)
Gateways Docker	9000/9001, 7700, 8000, 5678, 3003, 3001/5173	host	Interno (puglit-net)
Sidecars Python	5055, 8200	0.0.0.0	Interno

Binding a 0.0.0.0 es deliberado: en un RunPod Pod, el acceso externo va por el **proxy de RunPod** (`https://<podid>-<port>.proxy.runpod.net`), que requiere exponer el puerto en la config del pod. En una VM, en cambio, se accede por **túnel SSH** (`ssh -L 3000:localhost:3000`). El setup imprime ambas variantes al terminar.

Sobre el **modo de servir** (nota en `setup-gpu-box.sh` paso 7): `SERVE=prod` corre un build de producción (sin HMR/websocket, **robusto detrás del proxy**); `SERVE=dev` (default) corre `next dev` (hot reload, más liviano, pero **flaky detrás de un proxy**). Para sesiones largas detrás del proxy de RunPod se recomienda `prod`.

El **preview slot** (`serve-winner.sh`): el puerto 4311 es un único slot reusable — cada ganador del torneo que se construye libera lo que estuviera sirviéndose antes (`lsof -ti:$PORT | xargs kill`), de modo que toda app aparece en la MISMA URL estable. Se expone el puerto en RunPod una sola vez.

La red de gateways es una bridge dedicada `puglit-net`; Nango llega al Postgres del host vía `host.docker.internal`.

8. Monitoring

El monitoreo de la **plataforma** es deliberadamente liviano y basado en HTTP health checks; el monitoreo de **hardware** (GPU, VRAM, disco, uptime del pod) se delega a la **telemetría del propio RunPod** (dashboard del proveedor), que es la fuente de verdad para el estado del compute.

Health checks reales en el código:

- `GET /api/doctor` — verifica el cableado de modelos y la salud del stack. Es lo que `setup-gpu-box.sh` y `rebuild.sh` poolean tras arrancar (`curl -s ../api/doctor | jq .`) y el watchdog usa

como liveness (`http_code == 200`).

- **GET /health** en los sidecars (ScrapeGraphAI `{ok:true, model}`) — usado por `setup-scrapegraph.sh` para confirmar que el sidecar levantó.
- **Estado de jobs/torneos por API** — `test-tournament.sh` y `train-brain.sh` poolean `GET /api/genetic/tournament?status=<jid>` y `GET /api/job/<id>` siguiendo las fases en vivo (`status` , `phase`).
- **Watchdog server-side** (`rebuild.sh` , loop `puglit-sweep-loop`): cada 45s hace `curl /api/cron/sweep` para que los builds avancen aunque nadie tenga abierta la página `/build` (un build puede tardar horas; el usuario debe poder cerrar la pestaña y volver). Sin esto, los builds sólo progresan mientras un browser poolea.

Métricas de calidad de cada build se persisten como evidencia medida (no como "claim"): el QA gate de `build-local.mjs` escribe `tests / coverage / smoke / pages0k / pagesFail` vía `writeStatus()` , y la traza de llamadas LLM se puntúa (Cost/Latency/Reliability/Context) por `swarm-profile.ts` + `run-trace.ts` . Las métricas longitudinales del swarm viven en `puglit-metrics` .

9. Logging

El logging es **basado en archivos** (`/tmp/*.log`), pensado para sobrevivir a la caída del terminal web — todos los procesos largos se backgroundean con `nohup ... > /tmp/<x>.log 2>&1 &` :

Log	Proceso
<code>/tmp/ollama.log</code>	Ollama serve
<code>/tmp/puglit-dev.log</code> / <code>/tmp/puglit-prod.log</code>	Next dev / prod
<code>/tmp/build.log</code>	<code>npm run build</code> (con marcador <code>BUILD_DONE rc=\$?</code>)
<code>/tmp/rebuild.log</code>	Orquestación de <code>rebuild.sh</code> (con <code>SERVE_READY</code> final)
<code>/tmp/puglit-sweep.log</code>	Watchdog del cron sweep
<code>/tmp/scrapegraph.log</code> / <code>/tmp/scrapper.log</code>	Sidecars
<code>/tmp/train.log</code>	Batch de brain-training

El patrón es **disconnect-proof**: `rebuild.sh` está pensado para correrse como `nohup bash infra/rebuild.sh > /tmp/rebuild.log 2>&1 &` y seguirse con `tail -f` . El pool de Postgres registra errores con prefijo `[pg]` (`_pool.on("error", ...)`). El nivel de detalle del pipeline se ve en el stdout de `build-local.mjs` , que loguea cada ronda de tsc (archivos · errores · best), cada reparación de SQL, el veredicto de QA de la Reina, y el resultado del runtime gate (smoke test) con las páginas que fallan.

Para crashes de runtime en producción de las apps generadas, el spine trae el módulo de observabilidad con `SENTRY_DSN` (ver `secrets.example.env`), pero eso es del producto generado, no de la plataforma.

10. Backup Strategy

El principio rector está escrito en `web/lib/brain-sync.ts` y los scripts: el **almacén autoritativo vivo es el Postgres en la nube**; los snapshots a git/bucket son el backup *belt-and-suspenders*. El cerebro (diary, métricas, exemplars, skills evolucionadas, módulos cosechados, XP de agentes) es el aprendizaje acumulado y es lo que NO se puede perder al apagar un pod.

`infra/brain-snapshot.sh`

Exporta el cerebro completo (`GET /api/admin/brain` con `x-puglit-service`) a un JSON timestamped (`/tmp/brain-<TS>.json`) y, si hay un `BRAIN_REPO` configurado (un checkout git), lo copia a `latest.json` + `snapshots/brain-<TS>.json`, hace `git add -A && commit && push`. Sin repo git, sube a un bucket temporal como copia off-pod mínima. Se corre por cron o **después de cada run**.

```
> Snapshot the agents' brain → a durable backup (git repo or bucket).
> The LIVE authoritative store is the cloud Postgres; this is the belt-and-suspenders backup.
```

Qué exporta (`exportBrain()` en `brain-sync.ts`)

Dos clases de datos, tratadas distinto para que pods paralelos nunca pierdan ni se contradigan:

- **Append-only / aditivo** (`puglit_agent_diary`, `puglit_metrics`, `verified_exemplars`, `puglit_skill_rejects`) → se unieron por clave de contenido (CRDT grow-set). Escritores concurrentes sólo agregan.
- **Mutable / autoritativo** (`puglit_skills`, `puglit_modules`, XP de `puglit_agents`) → arbitrado por score objetivo: la skill activa por área es la del mayor **held-out validation score**; el módulo, la versión mayor; el XP, el máximo.

`train-brain.sh` llama a `brain-snapshot.sh` al final del batch de 100 ideas, snapshotando el cerebro pre-cargado a git automáticamente.

11. Disaster Recovery

La recuperación está diseñada para ser **segura de correr en cualquier momento** (idempotente, merge-aware), no un restore destructivo.

`infra/brain-restore.sh` — merge, no clobber

Bootstrapea un pod fresco/offline **mergeando** el último snapshot al DB vivo (`POST /api/admin/brain` con el JSON). El header es explícito: *"Safe to run anytime: it UNIONS additive learnings and arbitrates skills by validation score — it never clobbers a newer brain. Run it on pod start, after psql genetic.sql."*

El merge (`mergeBrain()` en `brain-sync.ts`) hace exactamente eso:

1. **Append-only** → `unionInsert()` chequea existencia por clave (`IS NOT DISTINCT FROM`) e inserta sólo lo nuevo.
2. **Skills** → unión de versiones (archivadas), luego `consolidateActiveSkills()` marca activa la del mayor `val_score` por área.

3. **Módulos** → `ON CONFLICT (name) DO UPDATE ... WHERE EXCLUDED.version > puglit_modules.version` (se queda con la versión mayor).
4. **XP de agentes** → `UPDATE ... xp=GREATEST(...) / level=GREATEST(...)` (nunca pierde niveles).

Resultado: un pod que estuvo offline mientras otro evolucionaba puede reincorporarse sin pisar nada ni introducir aprendizaje incongruente.

Resumable training

El batch de brain-training (`infra/train-brain.sh`) es **resumable por diseño**: lleva un archivo `DONE` (`/workspace/brain-train-done.txt`) y hace `grep -qxF "$name" "$DONE"` para saltar ideas ya hechas. Si el pod se cae a mitad del batch de 100 builds, se re-corre el mismo comando y retoma donde quedó. Cada torneo y cada build se poolean con timeouts generosos (~25 min torneo, ~30 min build) y el script tolera fallos individuales (`set -uo pipefail` , sin `-e`) para no abortar todo el batch por un build roto.

Recovery completo de un pod nuevo (secuencia)

```
git clone ... && cd puglit
bash infra/setup-gpu-box.sh          # Ollama + modelos + Postgres + schema + roster
psql ... -f web/sql/genetic.sql      # asegura el schema del cerebro
SVC=$PUGLIT_SERVICE_TOKEN BRAIN_REPO=/workspace/puglit-brain \
  bash infra/brain-restore.sh        # merge del último snapshot → cerebro recuperado
```

12. Upgrade Procedures

Los upgrades se hacen con un flujo **git-pull + rebuild disconnect-proof**, sin downtime planificado más allá del rebuild.

Upgrade de código (camino feliz)

```
cd /workspace/puglit && git pull
nohup bash infra/rebuild.sh > /tmp/rebuild.log 2>&1 &
```

`rebuild.sh` es el procedimiento canónico de upgrade en caliente:

1. Mata TODO lo que pueda conflictuar (builds viejos, otros rebuilds, servers stale, Ollama, copias del propio script salvo sí mismo).
2. `rm -rf .next` + `npm run build` con **Ollama abajo** (el modelo no está en RAM → build memory-safe, sin OOM).
3. Levanta el server de prod (`npm run start -p 3000 -H 0.0.0.0`) + Ollama council (`OLLAMA_MAX_LOADED_MODELS=1`) + watchdog.
4. Espera a que `/api/doctor` responda 200.
5. `POST /api/genetic/seed` — crea cualquier tabla/columna nueva (auth, scoping, embedding) y re-siembra los 75 agentes. Es **idempotente: preserva XP/level/diary**. Así "rebuild" alcanza para que login + proyectos por usuario funcionen sin pasos manuales.

Cambios de schema

Cuando un upgrade trae cambios de schema, el flujo es: `psql ... -f web/sql/genetic.sql` (y/o `puglit.sql`). Ambos archivos están escritos **idempotentes** (`CREATE TABLE IF NOT EXISTS`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`), así que re-correrlos es seguro y aditivo. El seed de la API hace la misma migración idempotente desde dentro de la app. En producción (Supabase, donde el rol de app no puede DDL) el SQL se corre a mano una vez.

Reset duro (cuando el working tree quedó sucio)

Para forzar el árbol al estado remoto antes de un rebuild:

```
cd /workspace/puglit
git fetch origin && git reset --hard origin/main
psql ... -f web/sql/genetic.sql          # re-asegura el schema
nohup bash infra/rebuild.sh > /tmp/rebuild.log 2>&1 &
```

Seguridad de datos en upgrades: el `reset --hard` toca SÓLO el código (el working tree git). Los **datos** (Postgres del cerebro, modelos de Ollama en el volume) NO viven en el árbol git, así que un reset no los toca. El seed y el `genetic.sql` son idempotentes y preservan XP/diary. Aun así, la regla de oro es correr `brain-snapshot.sh` ANTES de cualquier upgrade no trivial, para tener el punto de retorno (sección 10) y poder `brain-restore.sh` si algo sale mal (sección 11).

Endurecimiento de producción (una vez)

`infra/prod-env.sh` escribe secretos fuertes a `web/.env.local` de forma idempotente (`PUGLIT_SESSION_SECRET`, `PUGLIT_SERVICE_TOKEN`, `CRON_SECRET`), dejando intactas las keys ya presentes, para que la app deje de correr con los defaults de dev forgeables. Se corre una vez tras el primer deploy y luego se rebuildea.

Resumen operativo

Operación	Comando
Setup de pod nuevo	<code>bash infra/setup-gpu-box.sh</code>
Levantar gateways	<code>bash infra/setup-gateways.sh</code>
Sidecar de scraping	<code>bash infra/setup-scrapegraph.sh</code>
Upgrade en caliente	<code>git pull && nohup bash infra/rebuild.sh > /tmp/rebuild.log 2>&1 &</code>
Backup del cerebro	<code>SVC=... BRAIN_REPO=... bash infra/brain-snapshot.sh</code>
Recovery del cerebro	<code>SVC=... bash infra/brain-restore.sh</code>
Brain-training (resumable)	<code>SVC=... BRAIN_REPO=... nohup bash infra/train-brain.sh &</code>
Test E2E rápido	<code>bash infra/test-tournament.sh</code>
Servir el ganador	<code>bash infra/serve-winner.sh "Nombre" "qué hace"</code>
Build local completo	<code>cd web && node scripts/build-local.mjs</code>

Patrón operativo recomendado: **encender pod** → **trabajar** → **snapshot del cerebro** → **STOP el pod**.
 Se paga compute sólo encendido; el volume persiste los modelos y los snapshots a git persisten el aprendizaje.